

第三章--自下而上语法分析

自下而上语法分析方法的基本思想是移进—归约。归约是推导的逆过程。



回顾

- 有文法 G ，开始符号为 S ，如果有 $S \xRightarrow{*} x \beta y$ ，则 $x \beta y$ 是文法 G 的**句型**， x, y 是任意的符号串
- 如果有 $S \xRightarrow{*} x A y$ ，且有 $A \xRightarrow{+} \beta$ ，则 β 是句型 $x \beta y$ 相对于非终结符 A 的**短语**
- 如果有 $S \xRightarrow{*} x A y$ ，且有 $A \rightarrow \beta$ ，则 β 是句型 $x \beta y$ 相对于 $A \rightarrow \beta$ 的**直接短语**
- 位于**一个**句型最左边的直接短语称为**句柄**

- 假定 α 是文法 G 的一个句子，如果序列： $\alpha_n, \alpha_{n-1}, \dots, \alpha_0 (=S)$ 满足如下条件，则序列 $\alpha_n, \alpha_{n-1}, \dots, \alpha_0$ 是一个**规范归约**：
 - (1) $\alpha_n = \alpha$ 是给定的句子
 - (2) $\alpha_0 = S$ 是文法的开始符号
 - (3) 对任何 $i, 0 < i \leq n$, α_{i-1} 是从 α_i 经把**句柄**替换为相应文法产生式的左部符号而得到的。
- 规范归约是最右推导的逆过程，又称为**最左归约**。
- 最右推导又称**规范推导**，由规范推导所得到的句型称**规范句型**，规范推导的逆过程是规范归约。

回顾

例：设有文法G[S]:

$$(1) S \rightarrow aAcBe$$

$$(2) A \rightarrow b$$

$$(3) A \xRightarrow[rm]{(1)} Ab \xRightarrow[rm]{(2)} \xRightarrow[rm]{(3)} \xRightarrow[rm]{(4)}$$

$$(4) B \rightarrow d$$

使用最右推导：

因为S $\xRightarrow{rm} aAcBe \xRightarrow{rm} aAcde$

所以abbcde是文法G的句子。

aAbcde

abbcde



3.4.3 LR分析法

- **LR分析法**：L——从左向右扫描输入串，R——构造最右推导的逆过程
- 大多数用上下文无关文法描述的高级语言的语法成分可以用LR分析器来识别。
- **LR分析**：根据当前分析栈中的符号串和向右顺序查看输入串的 K ($K \geq 0$) 个符号就可以唯一确定分析的动作是移进还是归约。
- 向前查看0个符号，就是LR(0)分析方法，向前查看1个符号，就是LR(1)方法。

LR分析：采用移进-归约分析，严格的规范归约。

- 优点：

- 比其他“移进-归约”方法使用广泛，识别率高
- 能用LL(1)分析法分析的所有文法都能使用LR方法来进行分析。
- LR分析法在扫描输入串时就能发现其中的任何错误，并能准确地指出出错位置。

- 缺点：

- 手工构造分析表工作量太大。必须使用自动生成器。

1 LR分析概述

- 自下而上分析的关键问题是如何确定可归约串。
- LR方法中通过识别活前缀而求得句柄。
- LR方法是逐步归约栈顶的句柄来进行语法分析。



1) LR分析的基本思想

- 在规范归约的过程中，一方面用栈存放已移进和归约出的整个符号串，即记住“历史”；另一方面，LR分析器还要面对“现实”，即结合当前输入符号；再根据所用的产生式推测未来可能面临的输入符号，即对未来进行“展望”。当某符号串出现在栈顶时，根据历史展望和现实三方面的内容决定栈顶的符号串是否构成了句柄。

2) LR分析器的构成

文法符号栈： $X_1X_2\cdots X_m$ 是
目前已移进并归约出的句
型部分。其实它是多余的，
已经概括到状态里。

输入串

$a_1 \cdots a_i \cdots a_n \#$

X_m

分析器实际上是一个带有先进后出栈的确定的有穷自动机。分析栈用来存放状态，状态概括了从分析开始直到某一归约阶段的全部历史和展望资料，**根据栈顶状态和输入符号就可以唯一决定下一个动作。**

先放到栈中的初始状态和符号。

action goto

分析表

LR分析程序，又称**总控程序**。所有的LR分析器都是相同的。

分析表(分析函数)，分析表包括**动作表(ACTION)**和**状态转换(GOTO)表**两个部分，它们都可用二维数组表示。不同文法的分析表不同，同一个文法采用不同LR分析器时，分析表也不同。

分析栈，包括文法符号栈和相应的状态栈，它们均是先进后出栈。

(1) LR分析程序：总控程序根据分析表的内容来决定其下一步的处理动作，分析表是根据具体的文法按某种规则构造出来的。

(2) 分析栈：包括文法符号栈和状态栈。将历史和展望综合成状态。栈里的每个状态概括了从分析开始直到某一归约阶段的全部历史和展望资料，根据**栈顶状态和输入符号**就可以**唯一决定下一个动作**。

(3) 分析表的组成:

① 分析动作表Action是一个二维数组

符号 状态	a_1	a_2	...	a_t
S_0	$\text{action}[S_0, a_1]$	$\text{action}[S_0, a_2]$	...	$\text{action}[S_0, a_t]$
S_1	$\text{action}[S_1, a_1]$	$\text{action}[S_1, a_2]$	...	$\text{action}[S_1, a_t]$
...	...	...	...	...
S_n	$\text{action}[S_n, a_1]$	$\text{action}[S_n, a_2]$	...	$\text{action}[S_n, a_t]$

表中 $\text{action}[S_i, a_j]$, 指出如果当前栈顶为状态 S_i , 输入符号为 a_j 时应执行的动作。其动作有四种可能, 分别为: 移进(S)、归约(r)、接受(acc)、出错(error)。

② 状态转换表goto 也是一个二维数组

符号 状态	X_1	X_2	$\dots$	X_t
S_0	goto[S_0, x_1]	goto[S_0, x_2]	$\dots$	goto[S_0, x_t]
S_1	goto[S_1, x_1]	goto[S_1, x_2]	$\dots$	goto[S_1, x_t]
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
S_n	goto[S_n, x_1]	goto[S_n, x_2]	$\dots$	goto[S_n, x_t]

表中goto[S_i, x_j]指出状态为 S_i ，遇到 X_j 时应转到的下一状态。

显然：分析表定义了一个以文法符号为字母表的DFA

3) LR分析过程:

- 用三元式:

(状态栈, 符号栈, 输入符号串)

表示分析过程中状态栈, 符号栈, 输入符号串的变化

- 初始时, 将状态 S_0 和 $\#$ 进分析栈。三元式为:

$(S_0, \#, a_1a_2\dots a_n\#)$

- 任一时刻的三元式为:

$(S_0S_1\dots S_m, \#X_1X_2\dots X_m, a_ia_{i+1}\dots a_n\#)$

- 分析器的下一步动作是由栈顶状态 S_m 和当前面临的输入符号 a_i 唯一确定的。

根据 S_m 和 a_i 查action表, $\text{action}[S_m, a_i]$ 有4种情况:

- **移进**: 当前输入符号 a_i 移进符号栈, 将action表中指出的状态 S 进状态栈。

$$(S_0 S_1 \cdots S_m \textcircled{S}, \# X_1 X_2 \cdots X_m \textcircled{a_i}, a_{i+1} \cdots a_n \#)$$

- **归约**: 若产生式 $A \rightarrow \beta$ 的右端长度为 r , 则两个栈顶的 r 个元素同时出栈, A 进符号栈; 再根据 S_{m-r} 和 A 查goto表, $S = \text{goto}[S_{m-r}, A]$ 进状态栈。三元式变为:

$$(S_0 S_1 \cdots \textcircled{S_{m-r}} \textcircled{S}, \# X_1 X_2 \cdots X_{m-r} \textcircled{A}, a_i a_{i+1} \cdots a_n \#)$$

- **接受**: 分析成功, 终止分析。三元式不再变化。
- **出错**: 报告出错信息。三元式的变化过程终止。

例：文法的分析表

设文法为G[L]：

(1) $E \rightarrow E+T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (F)$

(6) $F \rightarrow i$

产生的
式的
序号

S_i 表示把当前输入
符号移进栈，第*i*个
状态进状态栈。

*i*表示转第*i*个状
态，即第*i*个状
态进状态栈。

表示按第*i*个产生
式进行归约

空白表示分
析动作出错

状态							GoTo		
	i						E	T	F
0	S_5				S_4		1	2	3
1		S_6				acc			
2		r_2	S_7		r_1				
3		r_4	r_4		r_1				
4				S_4					
5			r_6		r_6	r_6			
6	S_5			S_4		r_1		9	3
7	S_5			S_4					10
8		S_6			S_{11}				
9		r_1	S_7		r_1	r_1			
10		r_3	r_3		r_3	r_3			
11		r_5	r_5		r_5	r_5			

例：LR的具体分析过程：

设文法为G[L]：

(1) $E \rightarrow E+T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (F)$

(6) $F \rightarrow i$

为了介绍LR分析过程，直接给出该文法的分析表，以后再介绍如何生成

状态	ACTION						GoTo		
	i	+	*	(	)	#	E	T	F
0	S ₅			S ₄			1	2	3
1		S ₆				acc			
2		r ₂	S ₇		r ₂	r ₂			
3		r ₄	r ₄		r ₄	r ₄			
4	S ₅			S ₄			8	2	3
5		r ₆	r ₆		r ₆	r ₆			
6	S ₅			S ₄		r ₁		9	3
7	S ₅			S ₄					10
8		S ₆			S ₁₁				
9		r ₁	S ₇		r ₁	r ₁			
10		r ₃	r ₃		r ₃	r ₃			
11		r ₅	r ₅		r ₅	r ₅			

根据上述分析表，对输入串 $i * i + i$ 的分析过程如下：



序号	状态栈	符号栈	产生式	输入串	说明
1	0	#		$i * i + i \#$	0和#进栈
2	05	#i		$* i + i \#$	i和 S_5 进栈
3	03	#F	$F \rightarrow i$	$* i + i \#$	i和 S_5 退栈, F和 S_3 进栈
4	02	#T	$T \rightarrow F$	$* i + i \#$	F和 S_3 退栈, T和 S_2 进栈
5	027	#T*		$i + i \#$	*和 S_7 进栈
6	0275	#T*i		$+ i \#$	i和 S_5 进栈
7	027 <u>10</u>	#T*F	$F \rightarrow i$	$+ i \#$	i和 S_5 退栈, F和 S_{10} 进栈
8	02	#T	$T \rightarrow T * F$	$+ i \#$	$F * T$ 和 $S_{10}, 7, 2$ 退栈, T和 S_2 进栈
9	01	#E	$E \rightarrow T$	$+ i \#$	T和 S_2 退栈, E和 S_1 进栈
10	016	#E+		$i \#$	+和 S_6 进栈
11	0165	#E+i		#	i和 S_5 进栈
12	0163	#E+F	$F \rightarrow i$	#	i和 S_5 退栈, F和 S_3 进栈
13	0169	#E+T	$T \rightarrow F$	#	F和 S_3 退栈, T和 S_9 进栈
14	01	#E	$E \rightarrow E + T$	#	T+E和 $S_9, 6, 1$ 退栈, E和 S_1 进栈

- **LR方法**：根据具体文法的分析表对输入串进行分析处理。
- **LR分析过程**：在总控程序的控制下，从左到右扫描输入符号串，根据分析栈中的状态和当前输入符号，按分析表中的内容完成相应的分析工作。

问题

- 如何构造LR分析表？

2、LR(0) 分析

1) 基本概念

- 字的**前缀**：指该字的任意首部。
- **活前缀**：规范句型的一个前缀，不含句柄之后的任何符号。在它之后增添一些终结符号后，就成为规范句型。即：
- 对于文法G， 若 $\overset{*}{S} \Rightarrow a\beta$, $\beta \in V_t^*$ ，称a为**活前缀**。
- 在LR分析的过程中，假定输入串是一个句子，任何时候符号栈里的文法符号都构成活前缀，配上输入串的剩余部分，就成为规范句型。
- 构造一个有穷自动机来识别文法G的所有活前缀，这样就可以自动生成LR分析表。

序号	状态栈	符号栈	产生式	输入串	说明
1	0	#		i*i+i#	0和#进栈
2	05	#i		*i+i#	i和S ₅ 进栈
3	03	#F	F→i	*i+i#	i和S ₅ 退栈, F和S ₃ 进栈
4	02	#T	T→F	*i+i#	F和S ₃ 退栈, T和S ₂ 进栈
5	027	#T*		i+i#	*和S ₇ 进栈
6	0275	#T*i		+i#	i和S ₅ 进栈
7	027 <u>10</u>	#T*F	F→i	+i#	i和S ₅ 退栈, F和S ₁₀ 进栈
8	02	#T	T→T*F	+i#	F*T和S ₁₀ ,7,2退栈, T和S ₂ 进栈
9	01	#E	E→T	+i#	T和S ₂ 退栈, E和S ₁ 进栈
10	016	#E+		i#	+和S ₆ 进栈
11	0165	#E+i		#	i和S ₅ 进栈
12	0163	#E+F	F→i	#	i和S ₅ 退栈, F和S ₃ 进栈
13	0169	#E+T	T→F	#	F和S ₃ 退栈, T和S ₉ 进栈
14	01	#E	E →E+T	#	T+E和S ₉ , 6, 1退栈, E和S ₁ 进栈

- **LR(0) 项目**：在文法G中每个产生式的右部适当位置添加一个圆点构成项目。

- 例如：产生式 $S \rightarrow XYZ$ 对应有4个项目：

[0] $S \rightarrow \cdot XYZ$

[1] $S \rightarrow X \cdot YZ$

[2] $S \rightarrow XY \cdot Z$

[3] $S \rightarrow XYZ \cdot$

产生式 $A \rightarrow \varepsilon$ 只对应一个项目： $A \rightarrow \cdot$

- **项目集**：若干个项目组成的集合。 例如：对于上述产生式的4个项目即构成一个项目集。



练习

- 求文法： $S \rightarrow aS \mid bS \mid a$
的LR(0)项目集

$S \rightarrow aS$

$S \rightarrow bS$

$S \rightarrow a$

$S \rightarrow \cdot aS$

$S \rightarrow \cdot bS$

$S \rightarrow \cdot a$

$S \rightarrow a \cdot S$

$S \rightarrow b \cdot S$

$S \rightarrow a \cdot$

$S \rightarrow aS \cdot$

$S \rightarrow bS \cdot$

• **后继符号**：在项目中紧跟在符号“ $\cdot$ ”后面的符号称为该项目的后继符号。表示下一时刻遇到的符号。

后继符号有多种，**据此将项目分为多种**：

(1) **移进项目**：后继符号为终结符： $A \rightarrow \alpha \cdot a \beta$

(2) **待约项目**：后继符号为非终结符： $A \rightarrow \alpha \cdot B \beta$

(3) **归约项目**：后继符号为空：即圆点在最右边

$$A \rightarrow \alpha \cdot$$

(4) **接受项目**：归约项目的左边是文法开始符号组成的集合称为后继符号集。
后继符号集：项目集中各项的后继符号所组成的集合。

项目集 $\{ E \rightarrow E \cdot + T, F \rightarrow \cdot i \}$ 的后继符号集为 $\{ +, i \}$

2) 构造NFA的方法:

- 写出文法的所有项目，每个项目是一个状态
- 规定项目1为NFA的唯一初态
- 若状态i和状态j出自同一产生式，而且状态j的圆点只落后于状态i一个位置：
 - 若i的圆点后是a，从i到j画一条弧，标记为a
 - 若i的圆点后是A，则连两种弧：(1)从状态i画 ϵ 弧到所有的 $A \rightarrow \cdot \beta$ 的状态。(2)从状态i到j画弧，标记为A
 - 归约项目表示结束状态，用双圈表示

· 例：求文法对应的NFA

1) 该文法的项目有：

$$\begin{array}{lcl} S' & \rightarrow & E \\ E & \rightarrow & aA \mid bB \\ A & \rightarrow & cA \mid d \\ B & \rightarrow & cB \mid d \end{array}$$

1. $S' \rightarrow \cdot E$

2. $S' \rightarrow E \cdot$

3. $E \rightarrow \cdot aA$

4. $E \rightarrow a \cdot A$

5. $E \rightarrow aA \cdot$

6. $A \rightarrow \cdot cA$

7. $A \rightarrow c \cdot A$

8. $A \rightarrow cA \cdot$

9. $A \rightarrow \cdot d$

10. $A \rightarrow d \cdot$

11. $E \rightarrow \cdot bB$

12. $E \rightarrow b \cdot B$

13.

$E \rightarrow bB \cdot$

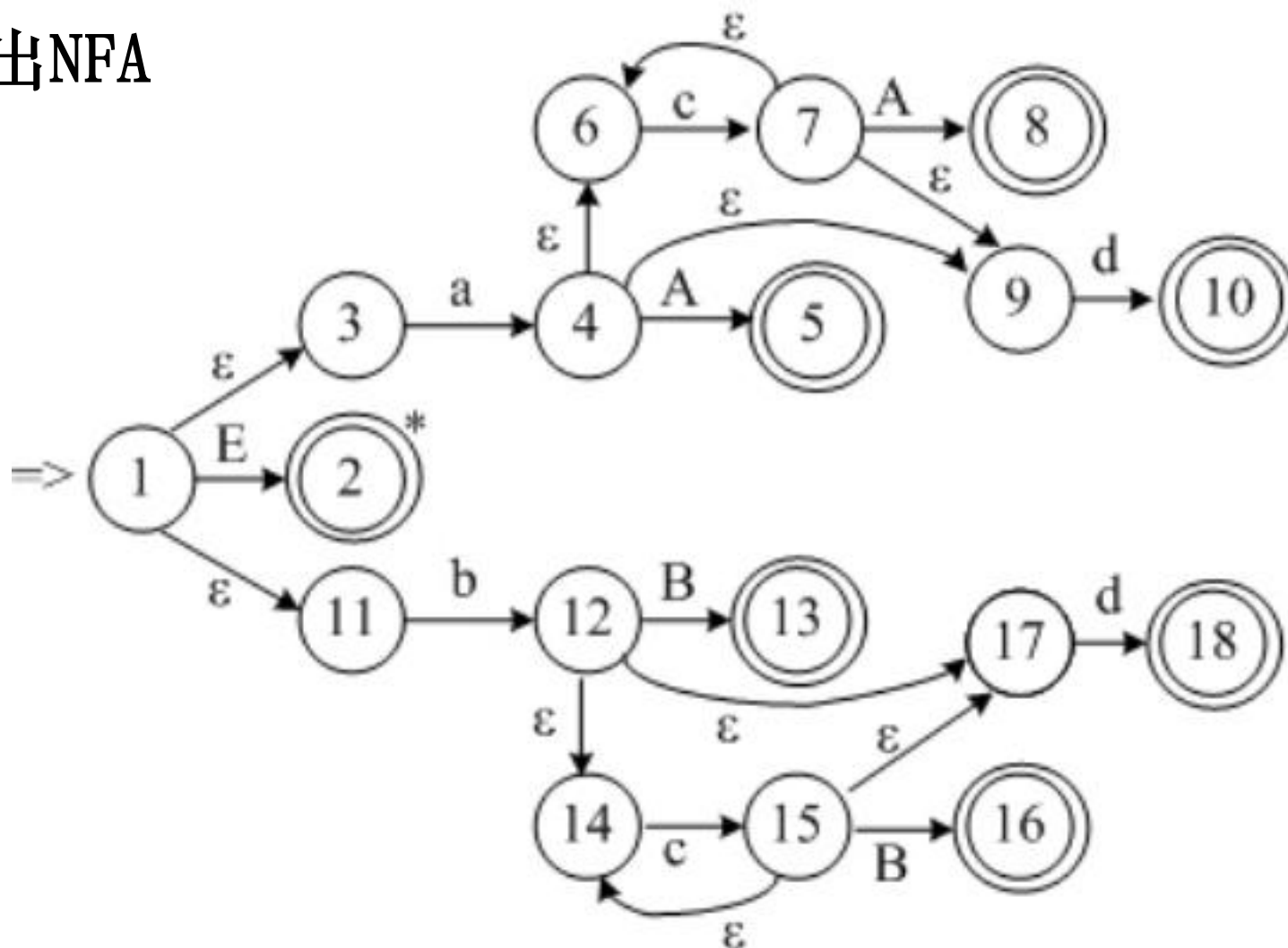
14. $B \rightarrow \cdot cB$

15. $B \rightarrow c \cdot B$

16.

$B \rightarrow cB \cdot$

2) 画出NFA



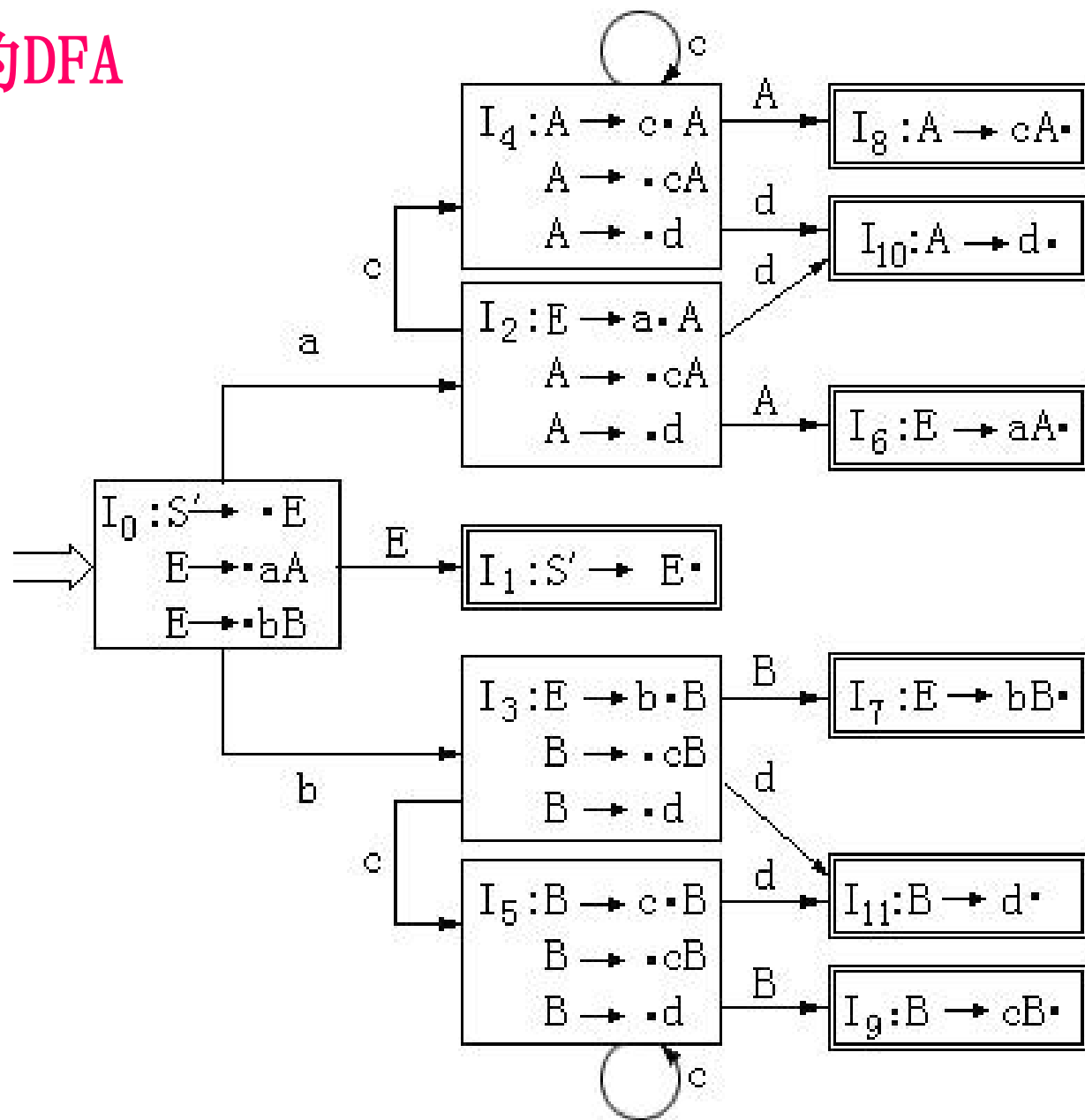
识别活前缀的NFA

2) 通过NFA构造识别活前缀的DFA:

- LR(0) 项目集规范族的定义：构成识别一个文法活前缀的DFA的项目集的全体
- 构造方法有两种：
 - 1) 先构造NFA，使用第二章的子集法将NFA确定化
 - 2) 直接使用闭包和状态转换函数进行计算

识别活前缀的DFA

使用子集法将NFA
确定化，得到一个
识别该文法的确定
的有穷自动机，其
每个状态是项目集



总结

LR 分析过程
LR(0)项目
LR(0)项目集规范族

